

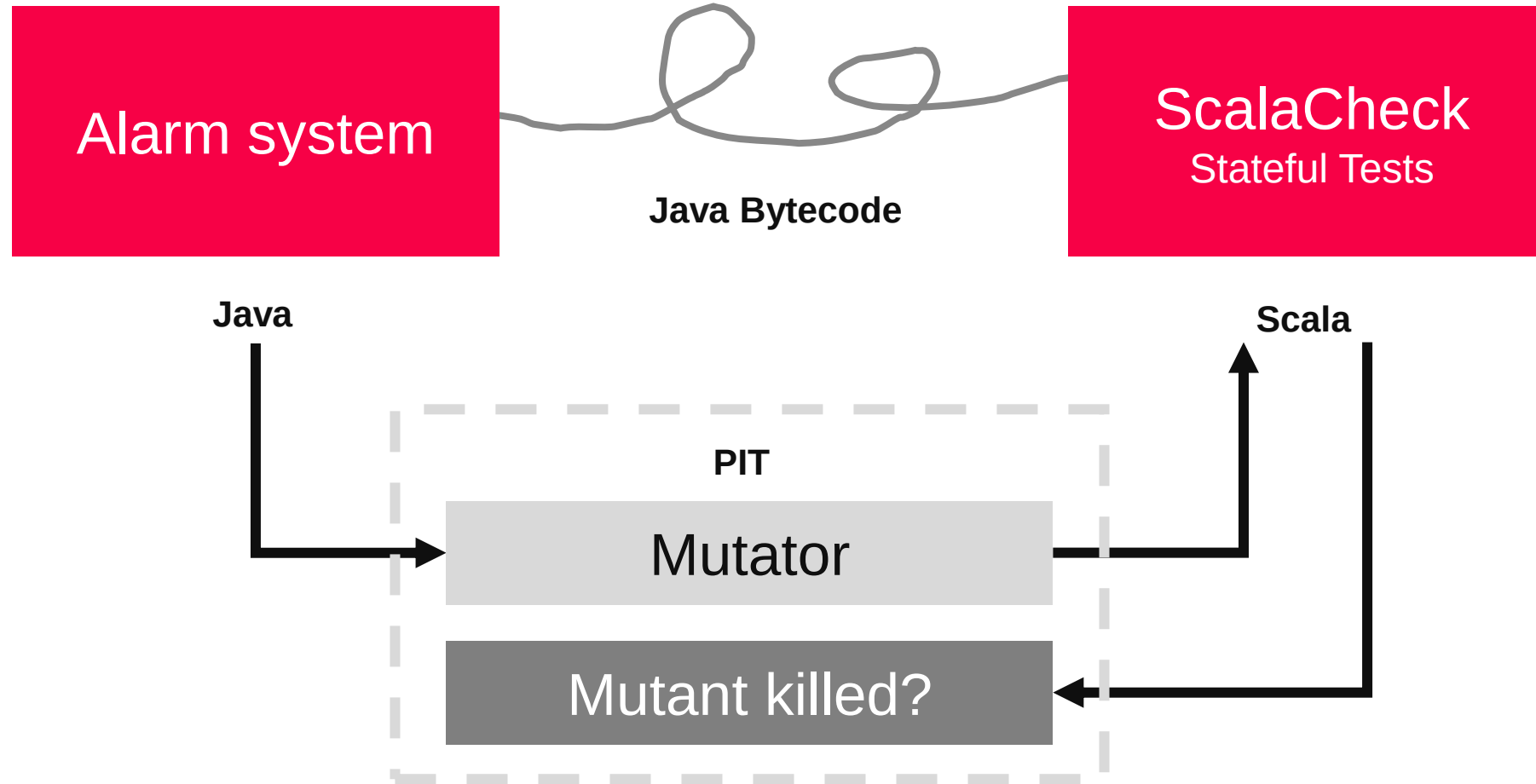
# Model-Based Mutation Testing with ScalaCheck and PIT

Anton Domonkos, Sebastian Albu  
Model Based Testing - Assignment Implementation Showcase

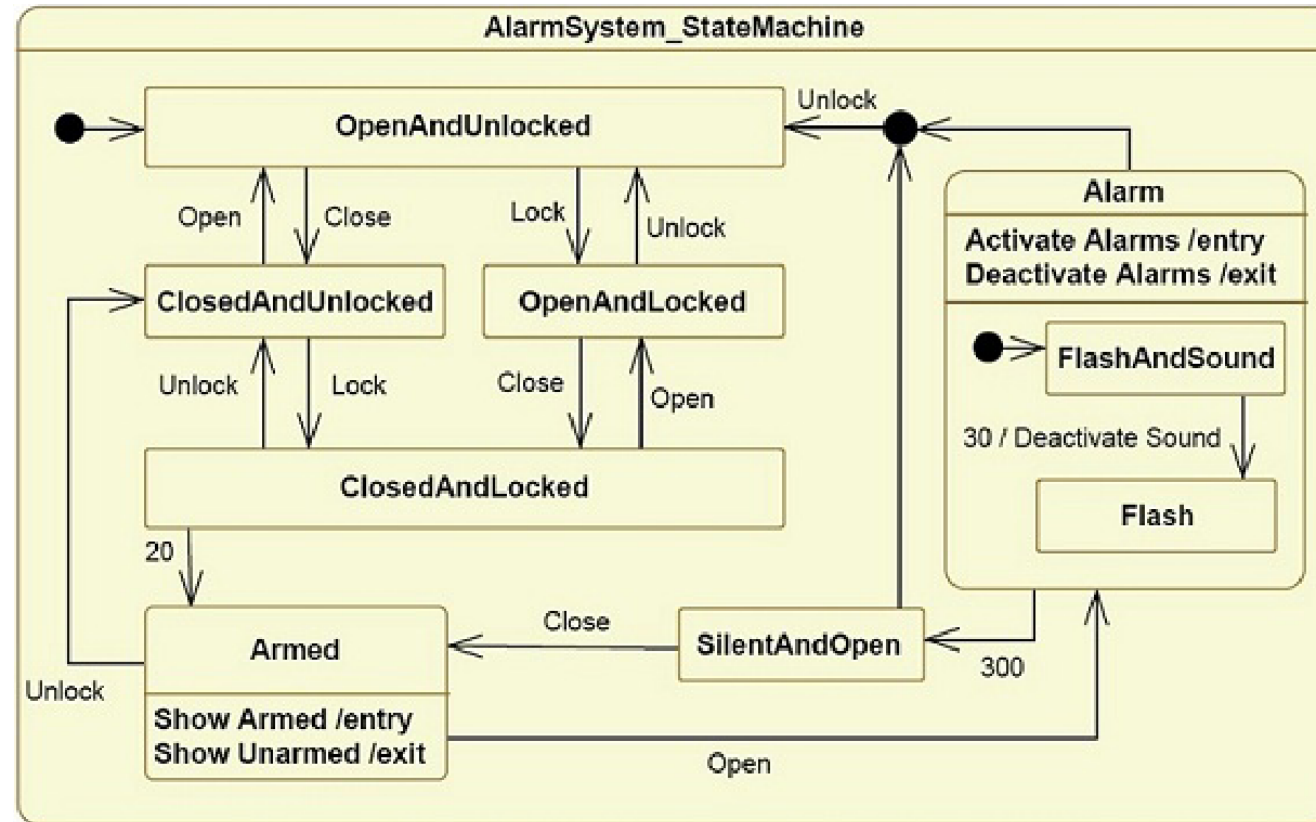
# Project Summary

- Implementing the alarm system in Java
- Modeling the alarm system in Scala
- Using ScalaCheck to verify implementation (*Model-Based Testing*)
- Using PIT to check model accuracy (*Mutation Testing*)

# Implementation Outline



# State Machine in Java



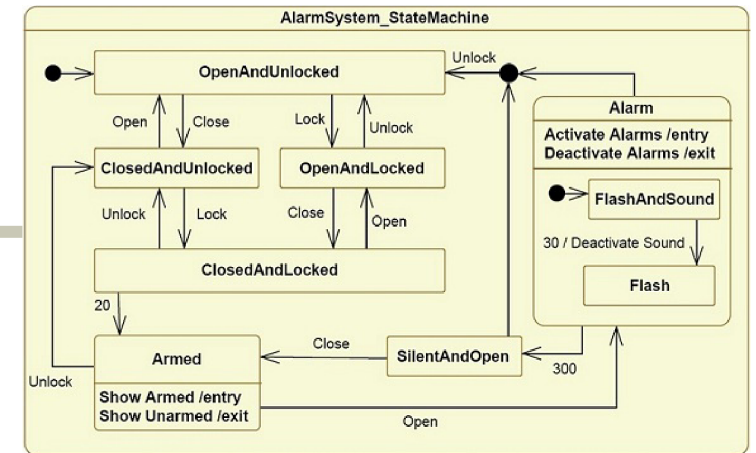
# State Machine in Java

- Translating main states from the basic FSM graph into *enum* values
- Each *enum* implements a transition method to react to input

```

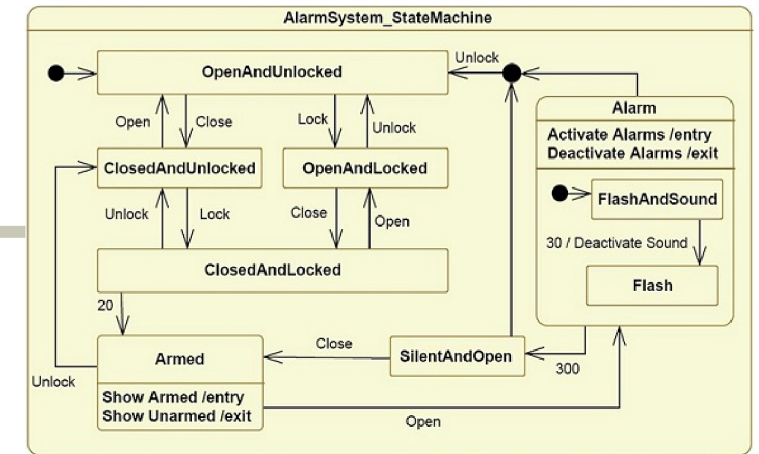
4  public enum SystemState implements MealyState {
64      Armed {
65          @Override
66          > public void onEntry(MealyLogger logger) { logger.println("Show Armed"); }
69
70          @Override
71          public SystemState next(MealyInput input) {
72              return switch (input) {
73                  case Unlock → ClosedAndUnlocked;
74                  case Open → Alarm;
75                  default → this;
76              };
77          }
78
79          @Override
80          > public void onExit(MealyLogger logger) { logger.println("Show Unarmed"); }
83      },
84

```



# State Machine in Java

- Translating main states from the basic FSM graph into *enum* values
- Each *enum* implements a transition method to react to input



```

4 public enum SystemState implements MealyState {
64     Armed {
65         @Override
66         > public void onEntry(MealyLogger logger) { logger.println("Show Armed"); }
69
70         @Override
71         public SystemState next(MealyInput input) {
72             return switch (input) {
73                 case Unlock → ClosedAndUnlocked;
74                 case Open → Alarm;
75                 default → this;
76             };
77         }
78
79         @Override
80         > public void onExit(MealyLogger logger) { logger.println("Show Unarmed"); }
83     },
84

```

38 usages Domonkos, Anton Ferenc

```

public enum MealyInput {
    4     Open,
    5     Close,
    6     Lock,
    7     Unlock,

```

# Extended State Machine in Java

```

3  public class StateMachine {
    11 usages
126  private boolean sendPinCode(String pin) {
127      if (currentState != SystemState.Armed) {
128          return true;
129      }
130      if (isValidPincode(pin) && pin.equals(PINCODE)) {
131          WRONG_PIN_COUNT = 0;
132          System.out.println("CORRECT PIN " + pin + " :");
133          return true;
134      }
135      WRONG_PIN_COUNT++;
136      System.out.println("WRONG PIN " + pin + " :(");
137      if (WRONG_PIN_COUNT == 3) {
138          System.out.println("3 TIMES WRONG - NOW ALARM ALAAAAARM");
139          setState(SystemState.Alarm);
140      }
141      return false;
142  }

```

**Requirement 4:** For unlocking, the car key sends a four-digit pincode. This pin is compared to an internally stored code. If the sent code fails to match for three times, then the alarm is triggered. This only happens when the car alarm system is armed.



Methods wrapping the 'original' state machine

# Time-dependent State Machine in Java

- Transition to new state
  - Reset tick counter
- Call *idle* command
  - Raise tick by one
  - Check if enough ticks passed to transition to a new state



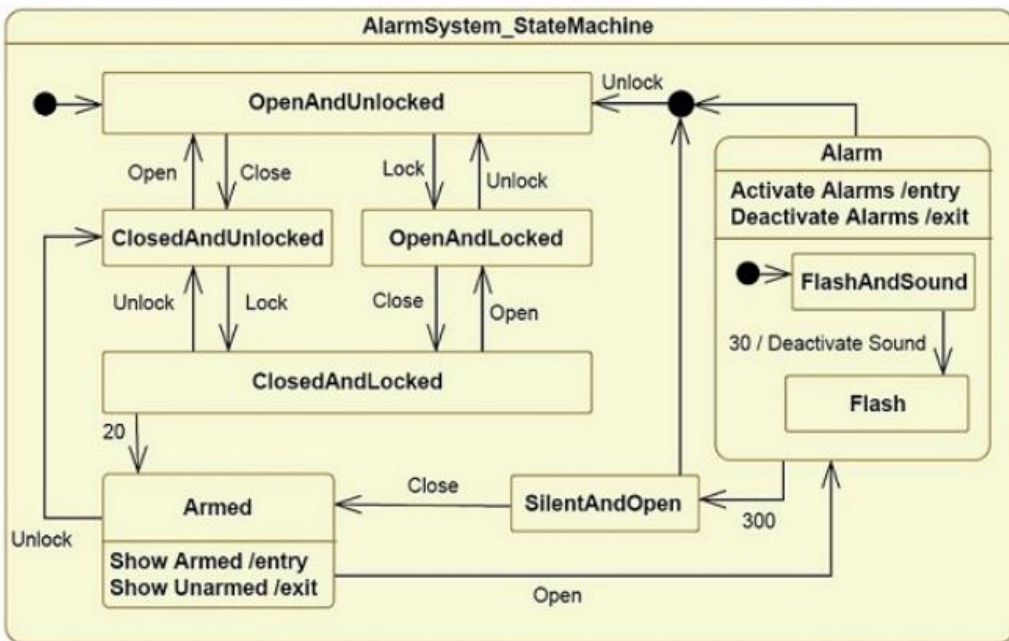
# Testing Implementation with JUnit

- Custom tests to verify specific command steps
- Tedious and manual process (bad)

```
146 Domonkos, Anton Ferenc
147 @Test
148 public void Test_CloseAllDoorsThenOpenLuggageBeforeArmed() {
149     fsm.lock();
150     fsm.closeDoor( doorIndex: 0);
151     fsm.closeDoor( doorIndex: 1);
152     fsm.closeDoor( doorIndex: 2);
153     fsm.closeDoor( doorIndex: 3);
154     fsm.closeBonnet();
155     fsm.closeLuggage();
156     fsm.openLuggage();
157     assert fsm.getState() == SystemState.OpenAndLocked;
158     fsm.closeLuggage();
159     fsm.idle( time: 1000);
160     assert fsm.getState() == SystemState.Armed;
161 }
```

# Stateful testing in Scala with ScalaCheck

- We used the ScalaCheck commands for the modelling of the `AlarmSystemSpecification` in Scala.
- `import org.scalacheck.commands.Commands`
  - Useful for testing stateful systems (systems where the behaviour is not solely based on in/outputs but also on the current system state). The used parameters for the Scala case class:



Domonkos, Anton Ferenc

```
case class ModelAlarmSystem(state: SystemState, door_0_open: Boolean, door_1_open: Boolean,
    door_2_open: Boolean, door_3_open: Boolean,
    luggage_open: Boolean, bonnet_open: Boolean,
    number_of_failed_pin_tries: Integer, current_pin: String,
    number_of_failed_reset_tries: Integer, is_only_luggage_unlocked: Boolean,
    last_transition_tick: Int, alarm_state: AlarmState)
```

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - Commands can easily be generated by extending **Command** trait and implementing different methods that symbolize actions on the SUT.
  - We basically build a State Machine with Scala Pattern Matching.

```
1 package at.tugraz.alarmsystem.test
2
3 import at.tugraz.alarmsystem.implementation.{AlarmState, MealyLoggerConsumer, StateMachine, SystemState}
4 import org.scalacheck.Prop.propBoolean
5 import org.scalacheck.commands.Commands
6 import org.scalacheck.{Gen, Prop}
7
8 import scala.util.{Random, Try}
9
10 object AlarmSystemSpecifcation extends Commands {
```

This is then our entire "Scala Model"

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - Generators (for Command parameters and choosing a Command):

The screenshot shows Scala code for stateful testing using ScalaCheck's `Commands` trait. Red arrows point from explanatory text to specific code elements.

```

26  override def genInitialState: Gen[State] = ModelAlarmSystem(SystemState.OpenAndUnlocked, door_0_open = true,
27      door_1_open = true, door_2_open = true, door_3_open = true, luggage_open = true, bonnet_open = true,
28      number_of_failed_pin_tries = 0, current_pin = "6969", number_of_failed_reset_tries = 0,
29      is_only_luggage_unlocked = false, last_transition_tick = 0, alarm_state = AlarmState.FlashAndSound)
30
31  new {
32      override def genCommand(state: State): Gen[Command] = Gen.oneOf(genIdleCommand, genResetPincodeCommand,
33          genUnlockCommand, genUnlockLuggageCommand, genLockCommand, genOpenDoor, genCloseDoor, genOpenLuggage, genCloseLuggage, genOpenBonnet, genCloseBonnet)
34
35      // -----
36
37      def genDoorIndex(): Gen[Int] = Gen.choose(min = 0, max = 3)
38
39      def genPincode(): Gen[String] = Gen.oneOf(t0 = "6969", t1 = "1234", tn = "4321", "a45", "hl67", "7p92", Random.nextString(4), Random.nextInt(9999).toString)
  
```

Our Model starts in this state

Random Parameters for Door index or Pincode

When testing sequences of these commands will be randomly chosen

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - Generators of states:
  - Example for the resetPincode:

```

125  def genResetPincodeCommand: Gen[ResetPinCommand] = for {
126    old_pin <- genPincode()
127    new_pin <- genPincode()  ← Randomly "generated" ;) variables
128  } yield ResetPinCommand(old_pin, new_pin)
129
130  case class ResetPinCommand(old_pin: String, new_pin: String) extends Command {
131    type Result = AlarmSystemResponse
132
133    override def run(sut: Sut): Result = {
134      runWithObserve(sut, () => sut.resetPincode(old_pin, new_pin))
135    }
136

```

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - State transitions with Pattern Matching and Case Guards (for `resetPin`):

```

195  override def nextState(state: State): State = {
196      var newState = state.copy()
197
198      newState = newState.state match {
199          case SystemState.Armed if state.current_pin != pincode && state.number_of_failed_pin_tries == 2
200              => newState.copy(state = SystemState.Alarm, number_of_failed_pin_tries = 0,
201                  last_transition_tick = 0, alarm_state = AlarmState.FlashAndSound)
202          case SystemState.Armed if state.current_pin != pincode
203              => newState.copy(number_of_failed_pin_tries = state.number_of_failed_pin_tries + 1, last_transition_tick = 0)
204          case SystemState.Armed if state.current_pin == pincode
205              => newState.copy(state = SystemState.ClosedAndUnlocked, number_of_failed_pin_tries = 0, last_transition_tick = 0)
206          // We only need pin when armed
207          case SystemState.SilentAndOpen | SystemState.OpenAndLocked | SystemState.Alarm
208              => newState.copy(state = SystemState.OpenAndUnlocked, last_transition_tick = 0,
209                  alarm_state = AlarmState.FlashAndSound)
210          case SystemState.ClosedAndLocked => newState.copy(state = SystemState.ClosedAndUnlocked, last_transition_tick = 0)
211          case _ => newState
212      }
213
214      newState
215  }

```

Pattern Matching of all States and Transitions

In the end we return the newly transitioned state

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - **Preconditions:**
  - You can specify preconditions for the commands, ensuring that they are only executed in valid states. This helps in creating realistic scenarios.
  - HOWEVER, for our model we did not add any preconditions as that lead to too few testcases and we settled for alternatives.
  - BUT one can use them, and they are surely helpful.
  - We decided to rather focus on:

# Stateful testing in Scala with ScalaCheck

- `import org.scalacheck.commands.Commands`
  - **Postconditions/Properties:**
  - We can check whether our state transitions and effects of the command have been successful, so an “assertion after everything”.

```

217  override def precondition(state: State): Boolean = true
218
219  override def postCondition(state: State, result: Try[Result]): Prop = {
220    val newState = nextState(state)
221    val reply = result.get
222
223    ("old vs new state [SendUnlockCommand]" |: newState.state == reply.state) &&
224    ("Show Unarmed [SendUnlockCommand]" |: (if (state.state != newState.state && state.state == SystemState.Armed)
225      reply.log.contains("Show Unarmed") else true)) &&
226    ("Activate Alarms [SendUnlockCommand]" |: (if (state.state != newState.state && state.state == SystemState.Armed && newState.state == SystemState.Alarm)
227      reply.log.contains("Activate Alarms") else true)) &&
228    ("Show Deactivating Alarms [SendUnlockCommand]" |:
229      (if (state.state != newState.state && state.state == SystemState.Alarm)
230        reply.log.contains("Deactivate Alarms") else true)) &&
231    ("old vs new AlarmState [SendUnlockCommand]" |: newState.alarm_state == reply.alarmState)
232  }
233

```

Unused preconditions (messed with testing)

Postconditions verifying state transitions successful

This is property-based Testing!

# Stateful testing in Scala with ScalaCheck

## ■ Generating Testcases

- Live Demo? Generating at least 2000 successful randomly generated Tests

```
3 import junit.framework.TestCase
4 import org.junit.Assert.assertTrue
5 import org.scalacheck.Test
6 import org.scalacheck.Test.Parameters
7 import org.scalacheck.util.ConsoleReporter
8
9  Domonkos, Anton Ferenc
10 class AlarmSystemSpecificationTest extends TestCase {
11    Domonkos, Anton Ferenc
12    def testAlarmSystemModelBased(): Unit = {
13        val params: Parameters => Parameters = p => p
14        .withTestCallback(ConsoleReporter(2))
15        .withMinSuccessfulTests(minSuccessfulTests = 2000)
16        val result: Boolean = Test.check(AlarmSystemSpecification.property())(params).passed
17        assertTrue(result)
18    }
19 }
```



# Model-Based Mutation Testing with PIT

## ■ Generating Mutants

- We used PIT and ran it like the following

```

32 tasks.test {
33     useJUnit()
34 }
35
36 tasks.register<Test>("testModel") {
37     filter {
38         includeTestsMatching("at.tugraz.alarmsystem.test.AlarmSystemSpecificationTest.*")
39     }
40     group = "test"
41     description = "Runs model-based tests for the alarm system"
42 }
43
44 val PITEST_THREADS = 4 // set according to your capabilities
45 val PITEST_OUT_FORMATS = listOf("HTML")
46
47 pitest {
48     targetClasses.set(setOf("at.tugraz.alarmsystem.implementation.*")) //by default "${project.group}.*"
49     excludedClasses.set(setOf("at.tugraz.alarmsystem.implementation.BrightnessManager",
50         "at.tugraz.alarmsystem.implementation.SoundManager", "at.tugraz.alarmsystem.implementation.Main"))
51     targetTests.set(setOf("at.tugraz.alarmsystem.test.AlarmSystemSpecificationTest")) //by default "${project.group}.*"
52     threads.set(PITEST_THREADS)
53     outputFormats.set(PITEST_OUT_FORMATS)
54     timestampedReports.set(false)
55 }

```

Set the number very high if possible

We specify which classes should be tested

We specify which classes should be ignored

Recommended to use multiple Threads.  
It takes a while to generate + test mutants!

```

1 plugins {
2     application
3     scala
4     id("info.solidsoft.pitest") version "1.15.0"
5 }

```



# Model-Based Mutation Testing with PIT

## Generating Mutants

- Example of a Mutant that was found + killed

### Door.java

```

1 package at.tugraz.alarmsystem.implementation;
2
3 public class Door {
4     private DoorState doorState;
5
6     public void open() {
7         this.doorState = DoorState.OPEN;
8     }
9
10    public void close() {
11        this.doorState = DoorState.CLOSED;
12    }
13
14    public DoorState getDoorState() {
15        return this.doorState;
16    }
17 }

```

### Mutations

15 1. replaced return value with null for at/tugraz/alarmsystem/implementation/Door::getDoorState → KILLED

### Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NEGATE\_CONDITIONALS
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

← We successfully killed a mutant ☺



# Model-Based Mutation Testing with PIT

## Overall Report

- Live Demo? (Takes a while to generate HTML)

### Pit Test Coverage Report

#### Package Summary

at.tugraz.alarmsystem.implementation

| Number of Classes | Line Coverage | Mutation Coverage | Test Strength |
|-------------------|---------------|-------------------|---------------|
| 6                 | 91% 202/223   | 77% 102/133       | 82% 102/125   |

PIT also helps for line coverage ☺  
(some unreachable dead code)

#### Breakdown by Class

| Name                              | Line Coverage | Mutation Coverage | Test Strength |
|-----------------------------------|---------------|-------------------|---------------|
| <a href="#">AlarmState.java</a>   | 40% 2/5       | 40% 2/5           | 67% 2/3       |
| <a href="#">Door.java</a>         | 100% 6/6      | 100% 1/1          | 100% 1/1      |
| <a href="#">MealyLogger.java</a>  | 100% 8/8      | 100% 2/2          | 100% 2/2      |
| <a href="#">MealyState.java</a>   | 75% 3/4       | 50% 1/2           | 100% 1/1      |
| <a href="#">StateMachine.java</a> | 92% 133/144   | 77% 79/102        | 81% 79/97     |
| <a href="#">SystemState.java</a>  | 89% 50/56     | 81% 17/21         | 81% 17/21     |

Report generated by [PIT](#) 1.15.0

# Our Findings

- Model-based testing breaks down testing into bite-size steps
  1. Define model command by command
  2. Test implementation
  3. Refine model
- Simultaneously modeling and implementing enabled a feedback-loop of improving our understanding of the specification
- With rigorous model definitions alone, we were able to already kill 2/3 of all mutants on 'first try'